Programmation FPGA avec Eclat : performances et chaîne de compilation libre

Dahmane Ilan

1 Juin 2026 - 31 Juillet 2026

Tuteur universitaire :
Eugene Asarin
Giulio Manzonetto

Tuteur de stage :
Loïc Sylvestre
Emmanuel Chailloux
Jocelyn Sérot

Table des matières

1 Introduction **3**

2 Etat de l'art **4**

3 Expérimentations **5**

4 Conclusion **6**

Références **7**

1 Introduction

Dans l'industrie informatique, les processeurs généralistes (CPU) constituent depuis plusieurs décennies la solution dominante pour l'exécution des programmes. Leur architecture est conçue pour offrir une grande flexibilité et pouvoir traiter une grande variété de tâches. Cependant, cette généralité limite leur efficacité lorsqu'il s'agit d'exploiter le parallélisme massif présent dans certaines applications.

Pour répondre à certains besoins spécifiques, les GPU (Graphics Processing Units) ont été développés initialement pour accélérer le rendu graphique. Leur architecture massivement parallèle les rend particulièrement efficaces pour les applications reposant sur de nombreuses opérations similaires appliquées sur de grands ensembles de données, comme le traitement d'image ou l'intelligence artificielle.

Cependant, toutes les applications ne se prêtent pas naturellement à ce modèle de calcul. Certaines nécessitent des architectures spécialisées capables d'exploiter un parallélisme différent, avec des contraintes fortes en termes de latence, de consommation énergétique ou de traitement temps réel. C'est dans ce contexte que les FPGA (Field Programmable Gate Array) constituent une solution intéressante grâce à leur capacité de reconfiguration et leur aptitude à implémenter des architectures matérielles spécialisées.

Cependant, leur programmation reste un domaine complexe nécessitant traditionnellement une description matérielle bas niveau. Le langage Eclat s'inscrit dans cette problématique en proposant une approche permettant de décrire des calculs matériels avec un niveau d'abstraction supérieur tout en conservant une traduction efficace vers des architectures FPGA.

Ce stage a pour objectif d'étudier les performances de différentes approches de programmation en Eclat, notamment à travers la comparaison de styles fonctionnels et impératifs via l'implémentation d'un processeur LC-3.

2 Etat de l'art

Les FPGA représentent une évolution majeure dans le domaine de la conception matérielle. Contrairement aux circuits intégrés spécifiques (ASIC), dont la fabrication nécessite des coûts de conception initiaux très importants, les FPGA permettent de configurer après fabrication une architecture matérielle adaptée à un besoin particulier. Cette flexibilité réduit considérablement les coûts et les délais de développement, notamment pour les systèmes nécessitant des évolutions fréquentes ou des productions en volumes limités.

La particularité principale des FPGA réside dans leur capacité de reconfiguration. Ils permettent d'implémenter directement des architectures matérielles spécialisées capables d'exploiter un fort parallélisme spatial. Cette caractéristique en fait des accélérateurs matériels particulièrement adaptés à certaines applications industrielles nécessitant de fortes performances, comme le traitement du signal, l'intelligence artificielle, le calcul scientifique ou encore les systèmes embarqués temps réel.

Cependant, l'exploitation de ces architectures nécessite traditionnellement l'utilisation de langages de description matérielle (HDL), tels que le VHDL ou le Verilog. Contrairement aux langages de programmation classiques, ces langages ne décrivent pas une succession d'instructions exécutées par un processeur, mais une architecture matérielle composée de composants logiques fonctionnant potentiellement en parallèle. Cette approche offre un contrôle très fin sur l'implémentation matérielle, mais introduit également une complexité importante et nécessite des connaissances spécifiques en conception numérique. Or, la programmation de FPGA devrait être à la portée de n'importe quelle développeur afin d'utiliser la carte comme bon lui semble sans préter attention aux détails architecturaux (est-ce vrai ? n'importe quel dev devrait pouvoir dev un FPGA ?).

Un des exemples les plus connus est HLS (High-level Synthesis) afin de se concentrer sur l'aspect algorithmique et n'ont pas sur l'architecture du circuit. Nous pouvons notamment écrire dans un sous ensemble de C/C++ (pointeur et allocation dynamique parfois exclus) afin que l'outil de synthèse de haut niveau parse votre programme et le synthétise en HDL. Mais cette approche à une limite. Le contrôle de l'architecture par le concepteur du programme est grandement limité et est délégué à l'outil de synthèse (le parallélisme est inférer etc... creuser un peu plus).

C'est dans cette perspective que s'inscrit Eclat. Ce langage propose un paradigme de programmation plus proche des langages logiciels traditionnels (OCaml-like) tout en conservant la possibilité de générer une architecture matérielle efficace. Le compilateur Eclat permet ainsi de traduire les programmes écrits dans ce langage en une description matérielle VHDL synthétisable, pouvant ensuite être implémentée sur un FPGA.

3 Expérimentations

L'objectif de ce stage est d'étudier l'impact du paradigme de programmation utilisé en Eclat sur l'architecture matérielle générée et ses performances. Nous comparons une approche fonctionnelle, basée sur l'immuabilité des données et la création de nouvelles structures, avec une approche impérative exploitant davantage le partage mémoire. pour cela j'ai d'abord du prendre en main quartus qui est l'outil pour synthétiser du HDL sur carte physique. J'ai commencé par allumer simultannémament un paquet de leds afin de comprendre la chaîne de compilation d'un langage de description matériel jusqu'à la production du .sof qui décrit le circuit et permet de flash sur la carte le circuit synthétiser par eclat.

- Introduire le processeur sur lequel les benchmarks sont faits - Parler du nombre de bloc logique utilisé pour les deux synthèse - Parler du temps en cycle - Parler de la facilité d'écriture - Insérer des benchs avec un programme équivalent mais synthétisé avec la vm OCaml en Eclat

4 Conclusion

Nous pouvons voir que les deux approches ont leur qualités et leur défauts, d'un côté nous avons de la performance temporelle pour le style fonctionnel qui permet de réduire les couts d'accès du à l'abscence d'accès concurrent (qui coute 1 cycle par accès) mais d'un autre côté nous avons une explosion du nombre de blocs logiques programmables utilisé en raison des nombreuses copies de bus de fils etc... De l'autre nous avons l'exacte opposé, en terme de complexité pire mais qui en terme de taille de circuit et de blocs logiques programmables est bien supérieur et permet de faire des programme bien plus gros. Une seconde explication passe par la logique d'implémentation des structure de données des deux styles. Les tableaux sont dans la BRAM (je crois car il y a aussi la SDRAM mais c'est une mémoire externe et c'est pour les grosses donneés non ?) de la carte ce qui ne limite pas sa taille tandis que les vecteurs sont des suites de blocs logiques programmables ce qui limite sa taille à 128kb (voir photo ou plutot donner une équivalent en terme de nombre d'entiers 32 bits, plus clair) hypothétique en attendant de finir les benchmarks et donc la conclusion.....

Références

insérer thèse de Loïc insérer le document sur le HLS